

OSSP-SKILL-WEEK4

Session7

Task1: Process Synchronization and Child Process Monitoring Using waitpid()

Source code:

```
satwika@SATWIK: ~  
GNU nano 7.2  
  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/types.h>  
#include <sys/wait.h>  
#include <stdlib.h>  
  
int main() {  
    pid_t child1, child2;  
    int status;  
  
    printf("Session 7 - Process Synchronization\n");  
    printf("Parent PID: %d\n\n", getpid());  
  
    child1 = fork();  
  
    if (child1 < 0) {  
        perror("fork failed");  
        return 1;  
    }  
  
    if (child1 == 0) {  
        printf("Child 1 started. PID: %d\n", getpid());  
        sleep(2);  
        printf("Child 1 completed.\n");  
        exit(10);  
    }  
  
    child2 = fork();  
  
    if (child2 < 0) {  
        perror("fork failed");  
        return 1;  
    }  
  
    if (child2 == 0) {  
        printf("Child 2 started. PID: %d\n", getpid());  
        sleep(4);  
        printf("Child 2 completed.\n");  
        exit(20);  
    }  
  
    printf("Parent is monitoring both child processes...\n");  
  
    waitpid(child1, &status, 0);  
  
    if (WIFEXITED(status)) {  
        printf("Parent: Child 1 (PID %d) exited with status %d.\n",  
            child1, WEXITSTATUS(status));  
    }  
  
    waitpid(child2, &status, 0);  
  
    if (WIFEXITED(status)) {  
        printf("Parent: Child 2 (PID %d) exited with status %d.\n",  
            child2, WEXITSTATUS(status));  
    }  
  
    printf("Parent: All child processes completed.\n");  
  
    return 0;  
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^X Execute

Output:

```
satwika@SATWIK:~$ gcc session7_task1.c -o session7_task1  
satwika@SATWIK:~$ ./session7_task1  
Session 7 - Process Synchronization  
Parent PID: 551  
  
Parent is monitoring both child processes...
```

```
Child 1 started. PID: 552
Child 2 started. PID: 553
Child 1 completed.
Parent: Child 1 (PID 552) exited with status 10.
Child 2 completed.
Parent: Child 2 (PID 553) exited with status 20.
Parent: All child processes completed.
satwika@SATWIKA:~$
```

Observation: I learned how to synchronize and monitor child processes using `waitpid()`. I understood how a parent process can wait for specific child processes to complete and check their exit status using `WIFEXITED()` and `WEXITSTATUS()`. I also learned how process synchronization ensures that the parent correctly monitors the execution and completion of its child processes.

Task2: To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

Source code:

```
satwika@SATWIKA: ~
GNU nano 7.2

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

#define MAX_PATHS 100
#define MAX_LENGTH 4096

int main() {
    char *path_env;
    char *path_copy;
    char *directory;
    char full_path[MAX_LENGTH];
    char command[100];

    printf("Session 7 - PATH Command Resolution\n");

    path_env = getenv("PATH");

    if (path_env == NULL) {
        printf("PATH variable not found.\n");
        return 1;
    }

    path_copy = malloc(strlen(path_env) + 1);

    if (path_copy == NULL) {
        perror("malloc failed");
        return 1;
    }

    strcpy(path_copy, path_env);

    printf("\nEnter command to locate: ");
    scanf("%99s", command);

    directory = strtok(path_copy, ":");

    while (directory != NULL) {

        snprintf(full_path, sizeof(full_path),
                 "%s/%s", directory, command);

        if (access(full_path, X_OK) == 0) {
            struct stat file_info;

            if (stat(full_path, &file_info) == 0) {
                printf("\nExecutable found!\n");
                printf("Command      : %s\n", command);
                printf("Full path  : %s\n", full_path);
                printf("Executable permission: Yes\n");
                free(path_copy);
                return 0;
            }
        }

        directory = strtok(NULL, ":");
    }

    printf("\nCommand not found in PATH.\n");
    printf("Command: %s\n", command);
}
```

```

    free(path_copy);
}

return 0;
}

```

^G Help ^O Write Out ^W Where Is ^K Cut ^X Execute

Output:

```

satwika@SATWIKA: ~
satwika@SATWIKA:~$ gcc session7_task2.c -o session7_task2
satwika@SATWIKA:~$ ./session7_task2
Session 7 - PATH Command Resolution

Enter command to locate: ls

Executable found!
Command      : ls
Full path    : /usr/bin/ls
Executable permission: Yes
satwika@SATWIKA:~$ ./session7_task2
Session 7 - PATH Command Resolution

Enter command to locate: hello

Command not found in PATH.
Command: hello
satwika@SATWIKA:~$

```

Observation: I learned how to retrieve and use the PATH environment variable to search for executable programs. I understood how to split the PATH into individual directories and construct possible executable paths. I also learned how to check whether a command exists and has execute permission using `access()`, and how to handle commands that are not found in the PATH.

Session8

Task1: To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

Source code:

```

satwika@SATWIKA: ~
GNU nano 7.2

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 500
#define MAX_OUTPUT 1000

void expand_variables(const char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1) {

        if (input[i] == '$') {
            i++;

            /* Support ${VARIABLE} */
            if (input[i] == '{') {
                i++;

                char variable[100];
                int k = 0;

                while (input[i] != '\0' &&
                    input[i] != '}' &&
                    k < 99) {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                if (input[i] == '}')
                    i++;

                char *value = getenv(variable);

```

```

        if (value != NULL) {
            for (int v = 0;
                value[v] != '\0' && j < MAX_OUTPUT - 1;
                v++) {
                output[j++] = value[v];
            }
        } else {
            printf("Undefined variable: %s\n", variable);
        }
    }
}

```

^G Help ^O Write Out ^W Where Is ^K Cut ^X Execute

satwika@SATWIK: ~

GNU nano 7.2

```

/* Support $VARIABLE */
else if (isalnum((unsigned char)input[i]) ||
        input[i] == '_') {

    char variable[100];
    int k = 0;

    while ((isalnum((unsigned char)input[i]) ||
        input[i] == '_') &&
        k < 99) {
        variable[k++] = input[i++];
    }

    variable[k] = '\0';

    char *value = getenv(variable);

    if (value != NULL) {
        for (int v = 0;
            value[v] != '\0' && j < MAX_OUTPUT - 1;
            v++) {
            output[j++] = value[v];
        }
    } else {
        printf("Undefined variable: %s\n", variable);
    }
}
/* Just '$' */
else {
    output[j++] = '$';
}
} else {
    output[j++] = input[i++];
}
}

output[j] = '\0';
}

```

^G Help ^O Write Out ^W Where Is ^K Cut ^X Execute

satwika@SATWIK: ~

GNU nano 7.2

```

int main() {
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Session 8 - Variable Expansion\n");
    printf("Enter text containing variables:\n");
    printf("myshell> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty input.\n");
        return 0;
    }

    printf("\nOriginal input:\n");
    printf("%s\n", input);

    expand_variables(input, output);

    printf("\nExpanded output:\n");
    printf("%s\n", output);

    printf("\nExpansion validation:\n");
    printf("Variable expansion completed successfully.\n");
}

```

```
    return 0;
}
```

^G Help

^O Write Out

^W Where Is

^K Cut

Output:

satwika@SATWIKA: ~

```
satwika@SATWIKA:~$ cd ~
satwika@SATWIKA:~$ nano session8_task1.c
satwika@SATWIKA:~$ gcc session8_task1.c -o session8_task1
satwika@SATWIKA:~$ ./session8_task1
Session 8 - Variable Expansion
Enter text containing variables:
myshell> echo $USER
```

```
Original input:
echo $USER
```

```
Expanded output:
echo satwika
```

```
Expansion validation:
Variable expansion completed successfully.
satwika@SATWIKA:~$ ./session8_task1
Session 8 - Variable Expansion
Enter text containing variables:
myshell> Home directory: $HOME
```

```
Original input:
Home directory: $HOME
```

```
Expanded output:
Home directory: /home/satwika
```

```
Expansion validation:
Variable expansion completed successfully.
satwika@SATWIKA:~$ ./session8_task1
Session 8 - Variable Expansion
Enter text containing variables:
myshell> Value: $NOTDEFINED
```

```
Original input:
Value: $NOTDEFINED
Undefined variable: NOTDEFINED
```

```
Expanded output:
Value:
```

```
Expansion validation:
Variable expansion completed successfully.
satwika@SATWIKA:~$
```

Observation: I learned how to detect variable references such as `$USER` and `$HOME` and expand them using environment variable values. I also learned how to handle undefined variables, update tokens after expansion, and support variable formats such as `$VARIABLE` and `${VARIABLE}`.

Task2: To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Source code:

satwika@SATWIKA: ~

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 200

typedef void (*builtin_func)(char *arg);

void builtin_help(char *arg);
void builtin_pwd(char *arg);
void builtin_cd(char *arg);
void builtin_exit(char *arg);
```

```
typedef struct {
    char *name;
    builtin_func func;
} Builtin;

Builtin builtins[] = {
    {"help", builtin_help},
    {"pwd", builtin_pwd},
    {"cd", builtin_cd},
    {"exit", builtin_exit}
};

int num_builtins = sizeof(builtins) / sizeof(Builtin);

void builtin_help(char *arg) {
    printf("\nAvailable built-in commands:\n");
    printf("  help  - Display available commands\n");
    printf("  pwd   - Display current directory\n");
    printf("  cd    - Change current directory\n");
    printf("  exit  - Exit the shell\n");
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^X Execute
```

```
satwika@SATWIK: ~
GNU nano 7.2

void builtin_pwd(char *arg) {
    char cwd[MAX_INPUT];

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("pwd failed");
    }
}

void builtin_cd(char *arg) {
    if (arg == NULL || strlen(arg) == 0) {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(arg) != 0) {
        perror("cd failed");
    }
}

void builtin_exit(char *arg) {
    printf("Exiting shell...\n");
    exit(0);
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^X Execute
```

```
satwika@SATWIK: ~
GNU nano 7.2

int main() {
    char input[MAX_INPUT];
    char *command;
    char *arg;

    printf("Session 8 - Built-in Command Dispatcher\n");
    printf("Type 'help' to see available commands.\n\n");

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        command = strtok(input, " ");
        arg = strtok(NULL, " ");

        int found = 0;

        for (int i = 0; i < num_builtins; i++) {
            if (strcmp(command, builtins[i].name) == 0) {
                printf("Dispatching built-in: %s\n", command);
                builtins[i].func(arg);
            }
        }
    }
}
```

```

        found = 1;
        break;
    }
}

if (!found) {
    printf("Invalid command: %s\n", command);
    printf("Type 'help' to see available builtins.\n");
}

return 0;
}

```

^G Help ^O Write Out ^W Where Is ^K Cut

Output:

```

satwika@SATWIK: ~
satwika@SATWIK:~$ nano session8_task2.c
satwika@SATWIK:~$ gcc session8_task2.c -o session8_task2
satwika@SATWIK:~$ ./session8_task2
Session 8 - Built-in Command Dispatcher
Type 'help' to see available commands.

myshell> help
Dispatching built-in: help

Available built-in commands:
  help  - Display available commands
  pwd   - Display current directory
  cd    - Change current directory
  exit  - Exit the shell

myshell> pwd
Dispatching built-in: pwd
/home/satwika

myshell> cd /tmp
Dispatching built-in: cd

myshell> pwd
Dispatching built-in: pwd
/tmp

myshell> hello
Invalid command: hello
Type 'help' to see available builtins.

myshell> exit
Dispatching built-in: exit
Exiting shell...
satwika@SATWIK:~$

```

Observation: I learned how to identify built-in commands and use a dispatch table to connect commands with their corresponding functions. I learned how built-in commands such as help, pwd, cd, and exit execute within the shell process. I also learned how to handle invalid commands and maintain shell state when commands such as cd change the current directory.